

FABLE_REBUILD_PLAN.md — Plano Mestre de Reconstrução (16 Fases)

Este é o documento mais importante do repositório. Ele guia toda a reconstrução do **Born to Run** de um projeto que **não compila** para um produto **estável, honesto, seguro e polido**. As fases estão ordenadas por dependência: **execute na sequência**. Não pule etapas — cada fase assume que as anteriores foram concluídas e validadas.

Princípios do plano

1. **Consolidar antes de estender.** Elimine duplicações antes de adicionar features.
2. **Schema é a fonte da verdade.** Alinhe o código ao `supabase/schema.sql` ; só mude o schema via migração planejada (Fase 2).
3. **Verde de verdade a cada fase.** `lint` , `typecheck` e `build` devem passar ao final de cada fase (a partir da Fase 1).
4. **Sem dados inventados.** Respeite os dados oficiais do `AGENTS.md` ; nunca fabrique contatos, números ou depoimentos.
5. **Mobile-first e acessível.** Toda tela nasce pensando em 375px e em WCAG AA.
6. **Commits pequenos**, em português, referenciando a fase (ex.: `feat(fase-3): design system Tailwind v4`).

Estado inicial (pré-Fase 1)

- Build/lint/typecheck **falham**.
- Duplicação de actions/rotas/componentes.
- Feed, fotos, logout, reset de senha, remover membro **quebrados**.
- Navegação pública por âncoras; galeria com imagens irrelevantes; contatos placeholder.

Convenção de cada fase

Cada fase traz: **Objetivo** • **Arquivos afetados** • **Alterações** • **Dependências** • **Riscos** • **Critérios de conclusão** • **Testes**.

FASE 1 — Arquitetura e Limpeza

Objetivo: fazer o projeto **compilar** e eliminar a duplicação estrutural, escolhendo uma única implementação correta por feature.

Arquivos afetados:

- `components/feed/PostCard.tsx` (corrigir export)
- `app/(dashboard)/dashboard/feed/page.tsx` (corrigir import)
- Remover: `app/actions/post.ts` , `app/actions/workouts.ts` , `app/actions/profile.ts` (após migrar o útil), `lib/supabase/middleware.ts`
- Consolidar rotas: manter `/dashboard/*` , remover `/treinos` , `/perfil` , `/fotos` da raiz do grupo
- Consolidar componentes: escolher entre `CreatePost` / `NewPostForm` e `PerfilForm` / `ProfileForm`

- Limpar imports/variáveis não usados (12 warnings de lint)
- Remover assets padrão do create-next-app em `public/`

Alterações:

1. Padronizar `PostCard` (definir named export único e ajustar todos os imports para o mesmo padrão).
2. Eleger `lib/actions/` como única fonte de server actions; apagar `app/actions/`.
3. Definir o conjunto único de rotas do aluno sob `/dashboard/*` e remover as rotas irmãs quebradas.
4. Escolher um componente por função e apagar o concorrente.
5. Remover código morto (`lib/supabase/middleware.ts`), imports não usados e assets órfãos.
6. Corrigir erros de lint (`no-explicit-any` , `no-unescaped-entities`).

Dependências: nenhuma (é a primeira fase).

Riscos: apagar a implementação errada e manter a certa exige atenção; risco de remover algo ainda referenciado. Mitigação: `grep` por imports antes de apagar cada arquivo.

Critérios de conclusão:

- `npm run build` **passa**.
- `npm run lint` sem erros (warnings zerados ou justificados).
- Não há duas actions/rotas/componentes para a mesma função.
- `grep` não encontra imports de arquivos removidos.

Testes:

- `npm run build && npm run lint && npx tsc --noEmit` verdes.
- Navegar por todas as rotas em `dev` sem erro de import.

FASE 2 — Banco de Dados e Segurança

Objetivo: tornar o schema coerente com o produto pretendido e fechar as brechas de RLS.

Arquivos afetados:

- `supabase/schema.sql`
- Nova pasta `supabase/migrations/` (migrações versionadas)
- `types/index.ts` (se o modelo de papéis mudar)

Alterações:

1. **Policy de DELETE em profiles** para admin (corrige “remover membro” — hoje falha silenciosa).
2. Adicionar **WITH CHECK** em `profiles_update_admin`.
3. **Decisão de papéis:** manter `member / admin` (recomendado) ou introduzir `treinador` no CHECK — decidir e documentar. Se mantiver, garantir que nenhum código dependa de `treinador`.
4. **Decisão de privacidade de treinos:** (a) manter treinos públicos a todos os autenticados (simples) **ou** (b) introduzir `assigned_to` + RLS por aluno. Documentar a escolha e migrar de acordo.
5. Gerar **tipos Database** do Supabase e passar a tipar os clients.
6. Estruturar migrações versionadas (deixar de depender de um único `schema.sql`).

Dependências: Fase 1 (código consolidado antes de mexer no contrato de dados).

Riscos: mudanças de schema podem quebrar queries; RLS mal ajustada pode expor/bloquear dados. Mitigação: aplicar em ambiente de teste do Supabase, validar cada policy com usuário member e admin.

CrITÉrios de conclusÃO:

- `deleteMember` remove de fato (com feedback de erro/sucesso).
- Políticas revisadas com `USING / WITH CHECK` corretos.
- Tipos `Database` integrados; queries com coluna errada passam a falhar em compile-time.
- Modelo de papéis e de privacidade **documentados** no `DATABASE_AND_SECURITY_REVIEW.md`.

Testes:

- Cenários RLS: member tenta ação de admin (deve falhar), admin remove membro (deve funcionar).
- `tsc` acusa erros ao usar coluna inexistente (prova de que a tipagem funciona).

FASE 3 — Design System

Objetivo: eliminar a quebra de estilo e criar um design system único e coerente em Tailwind v4.

Arquivos afetados:

- `app/globals.css`
- `tailwind.config.ts` (remover ou reativar via `@config` — decisão única)
- `app/layout.tsx` (fontes)

Alterações:

1. Decidir a via de config: **v4 no CSS** (recomendado) ou reativar `tailwind.config.ts` com `@config`.
2. Definir **todas** as classes utilitárias custom usadas nas páginas: `card`, `btn-primary`, `btn-secondary`, `btn-outline`, `badge` e variantes (`badge-red/green/orange/gray`), `section-title`, `section-subtitle`, `input-base`, `divider-*`, `shadow-card-lg`.
3. Consolidar a paleta em tokens (vermelho principal, verde secundário, laranja pontual, neutros claros) — ver `DESIGN_RECONSTRUCTION_BRIEF.md`.
4. Unificar tipografia (Inter para corpo, Barlow Condensed para títulos) e remover a referência a `Outfit`.
5. Garantir **light mode como padrão** (sem dark mode principal).

Dependências: Fase 1.

Riscos: definir classes com valores diferentes dos originais pode alterar o visual da home. Mitigação: comparar antes/depois em screenshots das páginas-chave.

CrITÉrios de conclusÃO:

- Nenhuma classe/variável CSS referenciada permanece indefinida.
- Todas as páginas internas e públicas renderizam com o estilo pretendido.
- Um único mecanismo de config Tailwind no projeto.

Testes:

- Inspeção visual de todas as páginas (mobile/tablet/desktop).
- Busca por classes indefinidas (`grep`) retorna vazio.

FASE 4 — Componentes Globais

Objetivo: reconstruir os componentes compartilhados (navegação, cards, formulários, estados) sobre o novo design system.

Arquivos afetados:

- `components/layout/Header.tsx` , `components/layout/Footer.tsx`
- Novos componentes de UI base (botão, input, card, badge, avatar, modal, toast/feedback)
- `components/feed/PostCard.tsx` , `components/workouts/WorkoutCard.tsx`

Alterações:

1. Reconstruir **Header** com navegação real (rotas, não âncoras) e menu mobile.
2. Reconstruir **Footer** com CTAs consistentes e contatos marcados como pendência (sem placeholders falsos publicados).
3. Criar biblioteca mínima de componentes base reutilizáveis.
4. Padronizar `PostCard` / `WorkoutCard` com o design system.

Dependências: Fases 1-3.

Riscos: navegação nova pode conflitar com âncoras da home. Mitigação: manter âncoras internas apenas para seções da home, usar rotas para páginas.

Critérios de conclusão:

- Header/Footer navegam para todas as páginas existentes.
- Componentes base documentados e reutilizados.

Testes:

- Clicar em todos os links do Header/Footer a partir de páginas diferentes.
- Verificar responsividade do menu mobile.

FASE 5 — Páginas Públicas

Objetivo: transformar o site em um multipágina coeso e honesto no conteúdo.

Arquivos afetados:

- `app/(public)/page.tsx` , `sobre/` , `equipe/` , `galeria/` , `resultados/` , `contato/`
- (Opcional) `app/(public)/historia/` ou redirect de `/historia` → `/sobre`

Alterações:

1. Corrigir navegação (âncoras → rotas) em todas as páginas.
2. **Galeria:** remover imagens irrelevantes/quebradas; usar só fotos reais da equipe; deixar placeholders claros onde faltarem fotos reais.
3. Substituir estatísticas fictícias pelos **dados oficiais** (~200 atletas, +200 participações, desde 2015).
4. Corrigir H1 da home (espaço entre palavras para leitores de tela).
5. Unificar CTAs (“Comece Agora” com destino único).
6. Contato: marcar telefone/e-mail como pendência do cliente; conectar o formulário a um backend real ou desabilitar com aviso honesto.

Dependências: Fases 3-4.

Riscos: conteúdo real insuficiente para preencher a galeria. Mitigação: usar placeholders honestos e solicitar mídia ao cliente.

Critérios de conclusão:

- Todas as páginas acessíveis pela navegação.

- Zero imagens quebradas ou fora de contexto.
- Zero dados inventados; contatos reais ou claramente pendentes.

Testes:

- Percorrer o site inteiro em 3 viewports.
- Verificar que nenhuma imagem retorna erro (`naturalWidth > 0`).

FASE 6 — Autenticação

Objetivo: deixar o fluxo de auth completo e sem pontas soltas.

Arquivos afetados:

- `app/(auth)/login/`, `cadastro/`, `recuperar-senha/`, `nova recuperar-senha/nova/`
- `lib/actions/auth.ts`
- `app/(dashboard)/layout.tsx` (logout), `app/(admin)/layout.tsx`
- `middleware.ts` (migrar para `proxy`; reforçar admin)

Alterações:

1. **Logout** via `logout()` de `lib/actions/auth.ts` no dashboard e admin (remover `/auth/signout`).
2. **Reset de senha:** corrigir `redirectTo` para o domínio do app e **criar** a rota `/recuperar-senha/nova`.
3. Conectar “Esqueceu a senha?” ao `/recuperar-senha`; implementar ou remover “Lembrar de mim”.
4. Centralizar login/cadastro nas actions (remover uso duplicado do client direto).
5. Migrar `middleware.ts` para a convenção `proxy` (Next 16) e reforçar checagem de admin em `/admin` (defesa em profundidade).

Dependências: Fases 1-2.

Riscos: mudança de middleware pode afetar sessão. Mitigação: testar login/logout e refresh de token.

Critérios de conclusão:

- Login, cadastro, logout, reset e definição de nova senha funcionam de ponta a ponta.
- Middleware sem aviso de descontinuação; admin protegido em duas camadas.

Testes:

- Fluxo completo: cadastrar → logout → login → esqueci senha → e-mail → nova senha.
- Acesso a `/admin` como member (bloqueado) e como admin (liberado).

FASE 7 — Área do Aluno

Objetivo: reconstruir o dashboard do aluno com as rotas consolidadas e as actions corretas conectadas.

Arquivos afetados:

- `app/(dashboard)/dashboard/page.tsx`, `feed/`, `treinos/`, `perfil/`
- `app/(dashboard)/layout.tsx` (sidebar/nav)
- `components/feed/*`, `components/profile/*`

Alterações:

1. Definir o **dashboard inicial** do aluno (resumo: próximos treinos, últimos comunicados, atalho ao

feed).

2. Corrigir a **sidebar** para apontar apenas para rotas válidas (`/dashboard` , `/dashboard/feed` , `/dashboard/treinos` , `/dashboard/perfil` , comunicados).
3. Conectar o **perfil** ao formulário único (Fase 1) com upload de avatar.

Dependências: Fases 1-6.

Riscos: rotas antigas ainda referenciadas em links. Mitigação: grep por `/treinos` , `/fotos` , `/perfil` (raiz) e substituir.

Critérios de conclusão:

- Navegação do aluno 100% funcional, sem rotas quebradas.
- Perfil salva e exibe dados/avatar corretamente.

Testes:

- Login como aluno → percorrer todas as abas → editar perfil → recarregar.

FASE 8 — Mini Rede Social (Feed)

Objetivo: feed social funcional de ponta a ponta.

Arquivos afetados:

- `app/(dashboard)/dashboard/feed/page.tsx`
- `components/feed/PostCard.tsx` , componente único de criação de post, comentários
- `lib/actions/feed.ts`

Alterações:

1. Listar posts com autor, foto, legenda, métricas (distância/tempo/pace), contagem real de likes/comentários e estado “curti”.
2. Conectar **criar post** (com upload), **curtir/descurtir**, **comentar** e **excluir** às actions de `lib/actions/feed.ts` .
3. Tratar estados vazios (sem posts) e de carregamento.

Dependências: Fases 2, 3, 7.

Riscos: contagem de likes/comentários exige agregação; sem tipos `Database` reintroduz bugs. Mitigação: usar contagens do Supabase e tipos gerados (Fase 2).

Critérios de conclusão:

- Publicar, curtir, comentar e excluir funcionam e refletem na UI (revalidação).

Testes:

- Publicar com/sem foto; curtir e descurtir; comentar; excluir (dono e admin).
- Verificar RLS: aluno não exclui post de outro (a menos que admin).

FASE 9 — Treinos

Objetivo: listagem e (conforme decisão da Fase 2) distribuição de treinos.

Arquivos afetados:

- `app/(dashboard)/dashboard/treinos/page.tsx`

- `components/workouts/*`
- `lib/actions/admin.ts` (criação por admin)

Alterações:

1. Listar treinos por nível (iniciante/intermediário/avançado) e data agendada.
2. Se a Fase 2 optar por privacidade: filtrar por `assigned_to` ; caso contrário, exibir treinos da equipe com aviso honesto (não prometer privacidade que não existe).
3. Remover qualquer texto/lógica de papel `treinador` inexistente.

Dependências: Fases 2, 7.

Riscos: a UI atual promete privacidade que o schema não garante. Mitigação: alinhar copy ao modelo real decidido na Fase 2.

Critérios de conclusão:

- Aluno vê os treinos corretos conforme o modelo escolhido.
- Admin cria/edita/exclui treinos que aparecem para os alunos.

Testes:

- Admin cria treino → aluno vê; filtros por nível funcionam.

FASE 10 — Comunicados

Objetivo: entregar comunicados do treinador aos alunos.

Arquivos afetados:

- Nova `app/(dashboard)/dashboard/comunicados/page.tsx`
- `app/(admin)/admin/comunicados/page.tsx`
- `lib/actions/admin.ts`

Alterações:

1. Criar a **tela de comunicados do aluno** (lista com título, conteúdo, data, autor).
2. Integrar os comunicados ao dashboard inicial (últimos avisos).
3. Garantir que `/dashboard/comunicados` exista (hoje retorna 404).

Dependências: Fases 3, 7.

Riscos: baixo.

Critérios de conclusão:

- Admin publica comunicado → aluno vê imediatamente (após revalidação).

Testes:

- Criar/excluir comunicado no admin → conferir na área do aluno.

FASE 11 — Painel do Treinador (Admin)

Objetivo: consolidar e polir o painel administrativo.

Arquivos afetados:

- `app/(admin)/*`, `components/admin/AdminForm.tsx`
- `lib/actions/admin.ts`

Alterações:

1. Garantir **remover membro** funcionando (após policy DELETE da Fase 2), com confirmação e feedback.
2. Revisar dashboard admin (contadores reais, atalhos).
3. Aplicar o design system aos formulários admin (hoje com classes indefinidas).
4. Remover import morto de `logout`.

Dependências: Fases 2, 3, 6.

Riscos: exclusão destrutiva sem confirmação. Mitigação: modal de confirmação + feedback de sucesso/erro.

Critérios de conclusão:

- CRUD completo de treinos, comunicados e membros com feedback adequado.

Testes:

- Promover/rebaixar/remover membro; criar/excluir treino e comunicado.

FASE 12 — PWA e Mobile

Objetivo: experiência mobile e PWA polida.

Arquivos afetados:

- `public/manifest.json`, ícones
- Layouts (navegação inferior mobile), `app/layout.tsx` (metadados/theme-color)

Alterações:

1. Revisar manifest (atalhos apontando para rotas válidas), theme-color alinhado à paleta.
2. Navegação inferior (bottom nav) no dashboard mobile.
3. Verificar instalabilidade (PWA) e comportamento offline básico, se aplicável.

Dependências: Fases 3–10.

Riscos: atalhos do manifest apontando para rotas removidas. Mitigação: revisar após consolidação de rotas.

Critérios de conclusão:

- App instalável; navegação mobile fluida; atalhos válidos.

Testes:

- Lighthouse PWA; teste de instalação; navegação em 375px.

FASE 13 — Acessibilidade

Objetivo: atingir conformidade prática com WCAG AA nos fluxos principais.

Arquivos afetados: páginas públicas e do dashboard, componentes base.

Alterações:

1. Corrigir contraste (ex.: estatísticas de `/resultados`).
2. Corrigir H1 sem espaço (leitores de tela).
3. Adicionar `alt` significativo, foco visível, navegação por teclado, `aria-*` onde necessário.
4. Garantir labels em todos os inputs.

Dependências: Fases 4–11.

Riscos: ajustes de contraste podem alterar a identidade. Mitigação: escolher tons que preservem a marca e passem no AA.

Critérios de conclusão:

- Auditoria de acessibilidade sem violações críticas nos fluxos-chave.

Testes:

- axe/Lighthouse a11y; navegação só por teclado; leitor de tela nos fluxos principais.

FASE 14 — Desempenho

Objetivo: otimizar carregamento e responsividade.

Arquivos afetados: uso de `next/image` , `imports` , `next.config.ts` , `queries`.

Alterações:

1. Otimizar imagens (`next/image` , tamanhos, formatos).
2. Revisar server vs client components (mover interatividade para client só onde necessário).
3. Reduzir over-fetching; paginar feed; usar índices já existentes.
4. Corrigir as 2 vulnerabilidades moderadas (postcss) quando viável sem breaking.

Dependências: Fases 5–12.

Riscos: `audit fix --force` pode quebrar. Mitigação: atualizar dependências com cautela e retestar build.

Critérios de conclusão:

- Métricas de performance (Lighthouse) satisfatórias em mobile.

Testes:

- Lighthouse performance; medir tempo de carregamento do feed com muitos posts.

FASE 15 — Testes

Objetivo: cobertura mínima confiável.

Arquivos afetados: nova infra de testes (unit/integração/e2e), CI.

Alterações:

1. Testes unitários para actions (`lib/actions/*`) e utils.
2. Testes de integração dos fluxos de auth/feed/treinos/comunicados.
3. Testes e2e dos caminhos críticos (cadastro→login→feed→logout; admin CRUD).
4. Pipeline de CI rodando `lint` + `typecheck` + `build` + testes.

Dependências: Fases 1-14.

Riscos: testes e2e dependem de ambiente Supabase. Mitigação: usar projeto de teste/mocks.

Critérios de conclusão:

- Suíte verde no CI; caminhos críticos cobertos.

Testes: a própria suíte + execução no CI.

FASE 16 — Deploy

Objetivo: publicar com segurança na Vercel.

Arquivos afetados: `README.md`, config Vercel, variáveis de ambiente, `.env.example`.

Alterações:

1. Configurar variáveis (`NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`) na Vercel.
2. Aplicar o schema/migrações no projeto Supabase de produção.
3. Definir Robson como `admin` (via SQL) após o primeiro login.
4. Validar build de produção, domínio, PWA e todos os fluxos em produção.
5. Atualizar documentação (README, este plano) com o estado final.

Dependências: Fases 1-15.

Riscos: variáveis erradas ou schema não aplicado quebram produção. Mitigação: checklist de deploy + smoke test pós-deploy.

Critérios de conclusão:

- App no ar, build de produção estável, fluxos verificados, sem segredos vazados.

Testes:

- Smoke test completo em produção (público + auth + dashboard + admin).

Ordem de prioridade (resumo)

1. **Fases 1-2** (destravar build + fundação de dados/segurança) — **bloqueantes**.
2. **Fases 3-4** (design system + componentes) — habilitam o resto.
3. **Fases 5-11** (features públicas e logadas) — o produto em si.
4. **Fases 12-14** (PWA/mobile, a11y, performance) — polimento.
5. **Fases 15-16** (testes + deploy) — confiabilidade e publicação.

Definição de “pronto” (Definition of Done) global

- `lint` + `typecheck` + `build` verdes.
- Sem duplicação de actions/rotas/componentes.
- Sem dados inventados; contatos reais ou pendências explícitas.
- RLS correta e testada; sem falhas silenciosas.
- Acessível (WCAG AA nos fluxos-chave) e mobile-first.
- Fluxos críticos cobertos por testes e verificados em produção.

Fim do FABLE_REBUILD_PLAN.md — plano mestre de reconstrução.